

Tarea 2 de Bases de Datos Distribuidas: Investigación

Reporte de Investigación: Sistemas y Arquitecturas de Bases de Datos Distribuidas

1. Evolución de las Arquitecturas: La Teoría Académica frente a la Realidad en la Nube

Al investigar los sistemas de bases de datos distribuidas, comprendí que es fundamental analizar cómo han cambiado los problemas que intentan solucionar. Mientras que la literatura clásica y la academia se centraban en la integración lógica de sistemas diferentes, hoy en día la industria está enfocada en lograr una escalabilidad física masiva en la nube.

A continuación, presento las diferencias clave que identifiqué:

- **Criterio de Clasificación:** La academia suele clasificar los sistemas basándose en su autonomía, heterogeneidad y distribución (el comportamiento lógico de los nodos). En contraste, en la industria actual (entornos Cloud-Native o NewSQL) la clasificación se hace según los recursos de hardware que se comparten, dominando actualmente el modelo "Shared Nothing" (nada compartido).
- **Topologías de Red:** Los modelos teóricos clásicos se basaban en redes Peer-to-Peer (P2P) y esquemas Cliente/Servidor. En la práctica moderna, esto evolucionó hacia modelos de replicación Maestro-Esclavo (arquitectura jerárquica) o Maestro-Maestro (arquitectura distribuida), donde es vital definir qué nodo tiene permisos de escritura y cuáles tienen permisos de lectura para evitar conflictos en los datos.
- **Integración de Sistemas:** Anteriormente, en los Sistemas Multibase de Datos (MDBS), el principal problema era usar "Wrappers" y "Mediadores" para poder traducir consultas entre bases distintas, como por ejemplo comunicar Oracle con SQL Server. Ahora, con los sistemas "Cloud-Native" desarrollados específicamente para la nube, el verdadero reto es poder escalar a nivel global para soportar a millones de usuarios simultáneos.

2. Estrategias de Distribución en los Gestores Modernos (DDBMS)

Para que cualquier sistema distribuido funcione de manera correcta, corroboré que debe implementar dos estrategias fundamentales tanto físicas como matemáticas:

1. **Fragmentación (Partitioning / Sharding):** Es el proceso de dividir la base de datos en pedazos, ya sea por filas o columnas, con el objetivo de repartir el peso del procesamiento y el almacenamiento entre múltiples servidores.

2. **Replicación:** Consiste en clonar estos fragmentos de datos en distintos lugares geográficos o nodos para asegurar que la información siga disponible incluso si un servidor sufre una falla.

Al analizar a los líderes del mercado, noté que implementan estas estrategias de formas muy distintas:

- **MongoDB (Gestor NoSQL Orientado a Documentos):**
 - Para fragmentar la información utiliza *Sharding*, apoyándose en enrutadores lógicos llamados "mongos" que se encargan de dirigir las consultas al servidor (shard) correcto.
 - Para la replicación usa *Replica Sets* bajo una arquitectura Maestro-Esclavo. En este modelo, un nodo Primario recibe todas las escrituras, mientras que los nodos Secundarios clonan los datos para respaldos o consultas de lectura.
- **Apache Cassandra (Gestor NoSQL de Alta Disponibilidad):**
 - Su replicación tiene una arquitectura *Masterless* (sin un maestro), siendo un modelo P2P puro donde todos los nodos son iguales. Usa un "Factor de Replicación" para asegurar que los datos existan en 'N' servidores; así, si uno se cae, los demás siguen respondiendo.
 - Para la fragmentación usa una topología de "Anillo" (Token Ring) que emplea algoritmos Hash (particionadores matemáticos) para repartir los datos equitativamente entre nodos virtuales.
- **CockroachDB (Gestor NewSQL Relacional):**
 - Este sistema permite fragmentar tablas particionándolas por "Rangos" o "Listas", lo que distribuye la carga a nivel global pero con la ventaja de mantener la estructura relacional y los JOINS.
 - Su replicación funciona a través de "Zonas de Replicación" manejadas mediante el protocolo de consenso Raft. Esto le permite garantizar que la información sea estrictamente consistente en todo momento, sin importar en qué lugar del mundo se encuentre el servidor.

3. El Teorema CAP: La regla fundamental de los sistemas distribuidos

Un dato adicional y vital en mi investigación es el Teorema CAP. Esta ley establece que en cualquier sistema informático distribuido es imposible garantizar al mismo tiempo estas tres características:

- **Consistencia (C):** Todos los nodos muestran exactamente la misma información al mismo tiempo.
- **Disponibilidad (A - Availability):** El sistema siempre responde, incluso si algunos de sus nodos se queman o fallan.
- **Tolerancia a Particiones (P):** El sistema sigue trabajando de manera normal aunque la red se caiga y los nodos pierdan comunicación entre sí.

La regla de oro dicta que **solo puedes elegir dos** de estas características.

- Si optas por una combinación **AP** (como Cassandra), tu sistema nunca se va a caer, pero existe el riesgo de que por unos milisegundos te muestre información desactualizada.
- Por otro lado, si eliges una combinación **CP** (como CockroachDB o MongoDB), tus datos siempre estarán perfectos, pero ante una falla grave en la red, el sistema bloqueará tus consultas para evitar que la información se corrompa.

4. Conclusión General

Tras realizar este análisis, concluyo que, aunque la teoría académica clásica define la replicación y la fragmentación de una forma muy abstracta, los modernos Sistemas de Gestión de Bases de Datos Distribuidas (DDBMS) combinan estos conceptos de una forma altamente pragmática para poder sobrevivir en la nube.

Soluciones como MongoDB apuestan por enrutadores lógicos y réplicas jerárquicas; gigantes como Cassandra se van por topologías P2P sin nodos maestros para priorizar la disponibilidad absoluta; y, finalmente, los gestores NewSQL como CockroachDB nos demuestran que sí es posible fragmentar información a nivel mundial conservando la consistencia matemática y robustez de una base de datos relacional tradicional como SQL Server.

Referencias bibliográficas

- Özsu, M. T., & Valduriez, P. (2011). *Principles of Distributed Database Systems* (3rd ed.). Springer. (Referencia principal para la arquitectura ANSI/SPARC, modelos P2P y MDBS).
- MongoDB, Inc. (2024). *MongoDB Documentation: Sharding and Replica Sets*. Recuperado de la documentación oficial de arquitectura de MongoDB.
- The Apache Software Foundation. (2024). *Apache Cassandra Architecture*. Recuperado de la documentación oficial del proyecto Cassandra (Topología en Anillo y Masterless).
- Cockroach Labs. (2024). *CockroachDB Architecture Overview: Replication and Distribution*. Recuperado de la documentación técnica oficial de CockroachDB (Protocolo Raft y NewSQL).
- Brewer, E. (2012). *CAP twelve years later: How the "rules" have changed*. Computer, 45(2), 22-29. (Referencia fundamental para agregar la explicación del Teorema CAP).
- Stonebraker, M. (1986). *The case for shared nothing*. Database Engineering Bulletin, 9(1), 4-9.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.
- Pavlo, A., & Aslett, M. (2016). *What's Really New with NewSQL?* ACM SIGMOD Record, 45(2), 45-55.